BÀI THỰC HÀNH THUẬT TOÁN TÌM KIẾM VÀ SẮP XẾP

Mục đích và Yêu cầu chung

Mục đích

Loạt bài tập này nhằm giúp người học:

- Hiểu rõ bản chất, cách thức hoạt động của các thuật toán sắp xếp (Sorting) và tìm kiếm (Searching) cơ bản đến nâng cao.
- Rèn luyện kỹ năng triển khai (implement) các thuật toán này bằng ngôn ngữ **Python 3**.
- Phân biệt được ưu, nhược điểm và trường hợp sử dụng phù hợp của từng thuật toán.
- Nâng cao tư duy giải quyết vấn đề, đặc biệt là tư duy đệ quy và tối ưu hóa hiệu năng.
- Học cách vận dụng các công cụ AI hiện đại (như ChatGPT, Claude, Copilot) như một người trợ giảng thông minh để gỡ lỗi, tối ưu code và khám phá sâu hơn về thuật toán.

Yêu cầu chung

- Tất cả các bài tập đều phải được viết bằng ngôn ngữ **Python (Phiên bản 3.x)**.
- Có thể chạy trực tiếp file dưới dạng Script Python (`.py`) hoặc sử dụng Jupyter Notebook/Google Colab để kiểm thử.
- Tập trung vào việc tự triển khai thuật toán (trừ khi có yêu cầu khác) thay vì chỉ sử dụng các hàm/phương thức có sẵn của Python (như `sort()`, `sorted()`, `min()`, `max()`).
- Viết code rõ ràng, tuân thủ chuẩn trình bày Python (PEP 8) và có chú thích (comment) ở những phần quan trọng.
- Tự chuẩn bị dữ liệu đầu vào (list số nguyên, danh sách đối tượng) để kiểm tra các trường hợp: danh sách rỗng, danh sách 1 phần tử, danh sách đã sắp xếp, danh sách sắp xếp ngược, danh sách có phần tử trùng lặp.

Bài 1: Người đưa thư tìm nhà

- **Mức độ:** Dễ
- **Thuật toán:** Tìm kiếm tuyến tính (Linear Search)

Mô tả yêu cầu chi tiết:

Viết một hàm `tim_tuyen_tinh(arr, x)` nhận đầu vào là một danh sách số nguyên `arr` (chưa sắp xếp) và một giá trị `x` cần tìm.

- Hàm này sẽ duyệt qua danh sách từ đầu đến cuối.
- Nếu tìm thấy `x` trong danh sách, hàm trả về chỉ số (index) đầu tiên của `x`.
- Nếu không tìm thấy `x`, hàm trả về `-1`.
- Viết khối mã chính (`if __name__ == "__main__":`) để tạo một danh sách và gọi hàm này để kiểm thử.

Gợi ý hướng dẫn:

1. Sử dụng vòng lặp `for` kết hợp hàm `range(len(arr))` để duyệt qua tất cả các chỉ số của danh sách.
2. Bên trong vòng lặp, so sánh phần tử hiện tại `arr[i]` với `x`.
3. Nếu bằng nhau, dùng `return i` để trả về chỉ số ngay lập tức.
4. Nếu vòng lặp kết thúc mà không tìm thấy, `return -1` ở cuối hàm.

Code mẫu (Giải pháp tham khảo):

Python

```python
def tim_tuyen_tinh(arr: list, x: int) -> int:
    for i in range(len(arr)):
        if arr[i] == x:
            return i  # Tìm thấy, trả về chỉ số
    return -1  # Không tìm thấy


if __name__ == "__main__":
    mang_so = [4, 11, 2, 9, 7, 22, 5]
    gia_tri_can_tim = 7
    ket_qua = tim_tuyen_tinh(mang_so, gia_tri_can_tim)

    if ket_qua == -1:
        print(f"Khong tim thay {gia_tri_can_tim} trong mang.")
    else:
        print(f"Tim thay {gia_tri_can_tim} tai chi so: {ket_qua}")
```

Vận dụng AI gia tăng trải nghiệm:

- **Hỏi AI:** "Hãy giải thích độ phức tạp thời gian (time complexity) của thuật toán tìm kiếm tuyến tính. Tại sao nó là $O(n)$?"

- **Yêu cầu AI:** "Hãy tạo cho tôi 5 bộ dữ liệu test (test case) khác nhau cho hàm tim_tuyen_tinh, bao gồm cả trường hợp tìm thấy, không tìm thấy, và danh sách rỗng."
- **Thử thách:** "Hãy viết lại hàm này sử dụng phương thức `list.index()` hoặc toán tử `in` trong Python và hỏi AI xem cách nào Pythonic hơn và tại sao."

Bài 2: Tra từ điển nhanh

- **Mức độ:** Trung bình
- **Thuật toán:** Tìm kiếm nhị phân (Binary Search)

Mô tả yêu cầu chi tiết:

Viết một hàm `tim_nhi_phan(arr, x)` nhận đầu vào là một danh sách số nguyên `arr` **ĐÃ ĐƯỢC SẮP XẾP TĂNG DẦN** và một giá trị `x` cần tìm.

- Hàm này phải triển khai thuật toán Tìm kiếm nhị phân (chia để trị).
- Nếu tìm thấy `x`, trả về chỉ số (index) của `x`.
- Nếu không tìm thấy, trả về `-1`.
- Trong chương trình chính, đảm bảo bạn gọi hàm `sort()` hoặc `sorted()` (hoặc hàm Bubble Sort từ Bài 3) để sắp xếp danh sách trước khi gọi hàm `tim_nhi_phan`.

Gợi ý hướng dẫn:

1. Sử dụng ba biến: `low` (chỉ số đầu), `high` (chỉ số cuối), và `mid` (chỉ số giữa).
2. Khởi tạo `low = 0` và `high = len(arr) - 1`.
3. Sử dụng vòng lặp `while low <= high`.
4. Tính `mid = low + (high - low) // 2` (dùng toán tử chia lấy phần nguyên `//` trong Python).
5. So sánh `arr[mid]` với `x`:
    - Nếu `arr[mid] == x`, trả về `mid`.
    - Nếu `arr[mid] < x`, `x` nằm ở nửa bên phải, gán `low = mid + 1`.
    - Nếu `arr[mid] > x`, `x` nằm ở nửa bên trái, gán `high = mid - 1`.
6. Nếu vòng lặp kết thúc mà không tìm thấy, trả về `-1`.

Code mẫu (Giải pháp tham khảo):

Python

```
def tim_nhi_phan(arr: list, x: int) -> int:
    low = 0
    high = len(arr) - 1

    while low <= high:
```

```
        mid = low + (high - low) // 2
        if arr[mid] == x:
            return mid  # Tìm thấy
        elif arr[mid] < x:
            low = mid + 1  # Tìm ở nửa bên phải
        else:
            high = mid - 1  # Tìm ở nửa bên trái

    return -1  # Không tìm thấy


if __name__ == "__main__":
    mang_so = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]  # Danh sách đã sắp xếp
    gia_tri_can_tim = 23
    ket_qua = tim_nhi_phan(mang_so, gia_tri_can_tim)

    if ket_qua == -1:
        print(f"Khong tim thay {gia_tri_can_tim} trong mang.")
    else:
        print(f"Tim thay {gia_tri_can_tim} tai chi so: {ket_qua}")
```

Vận dụng AI gia tăng trải nghiệm:

- **Hỏi AI:** "Tại sao điều kiện tiên quyết của Tìm kiếm nhị phân là danh sách phải được sắp xếp? Điều gì sẽ xảy ra nếu tôi chạy thuật toán này trên danh sách chưa sắp xếp?"
- **Gỡ lỗi:** Cố tình viết sai logic, ví dụ `low = mid` thay vì `low = mid + 1` và hỏi AI: "Code của tôi bị lặp vô hạn (infinite loop) trong một số trường hợp, hãy tìm lỗi giúp tôi."
- **Mở rộng:** "Làm thế nào để triển khai Tìm kiếm nhị phân bằng đệ quy (recursion) trong Python hoặc sử dụng thư viện chuẩn `bisect`?"

Bài 3: Xếp hàng theo chiều cao

- **Mức độ:** Dễ / Trung bình
- **Thuật toán:** Sắp xếp nổi bọt (Bubble Sort)

Mô tả yêu cầu chi tiết:

Viết một hàm `sap_xep_noi_bot(arr)` nhận đầu vào là một danh sách số nguyên và sắp xếp danh sách đó theo thứ tự tăng dần bằng thuật toán Bubble Sort.

- Thuật toán này hoạt động bằng cách lặp đi lặp lại việc duyệt qua danh sách, so sánh các phần tử liền kề và hoán đổi chúng nếu chúng sai thứ tự.

- Hàm này thay đổi trực tiếp danh sách đầu vào (sắp xếp "in-place").
- Viết chương trình chính để tạo một danh sách, gọi hàm sắp xếp, sau đó in danh sách ra màn hình để kiểm tra kết quả.

Gợi ý hướng dẫn:

1. Bạn sẽ cần hai vòng lặp lồng nhau.
2. Vòng lặp bên ngoài `i` chạy từ `0` đến `n - 2` (kiểm soát số lượt duyệt).
3. Vòng lặp bên trong `j` chạy từ `0` đến `n - i - 2` (vì các phần tử lớn nhất đã "nổi" về cuối danh sách sau mỗi lượt).
4. Trong vòng lặp `j`, so sánh `arr[j]` và `arr[j + 1]`. Nếu `arr[j] > arr[j + 1]`, hãy hoán đổi chúng bằng cú pháp hoán đổi ngắn gọn của Python: `arr[j], arr[j + 1] = arr[j + 1], arr[j]`.

Code mẫu (Giải pháp tham khảo):

Python

```python
def sap_xep_noi_bot(arr: list) -> None:
    n = len(arr)
    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                # Hoán đổi trực tiếp trong Python
                arr[j], arr[j + 1] = arr[j + 1], arr[j]


if __name__ == "__main__":
    mang_so = [64, 34, 25, 12, 22, 11, 90]
    print(f"Mang ban dau: {', '.join(map(str, mang_so))}")

    sap_xep_noi_bot(mang_so)

    print(f"Mang sau khi sap xep: {', '.join(map(str, mang_so))}")
```

Vận dụng AI gia tăng trải nghiệm:

- **Hỏi AI:** "Hãy tối ưu hóa thuật toán Bubble Sort trong Python. Điều gì xảy ra nếu danh sách đã được sắp xếp? Làm thế nào để thêm một cờ (flag `swapped`) để dừng sớm?"
- **Yêu cầu AI:** "Hãy mô tả trực quan (visualize) các bước của Bubble Sort với danh sách `[5, 1, 4, 2, 8]`."
- **So sánh:** "So sánh Bubble Sort với Insertion Sort (Sắp xếp chèn). Thuật toán nào tốt hơn trong thực tế?"

Bài 4: Chia để trị

- **Mức độ:** Khó
- **Thuật toán:** Sắp xếp nhanh (Quick Sort)

Mô tả yêu cầu chi tiết:

Viết chương trình Python triển khai thuật toán Quick Sort để sắp xếp một danh sách số nguyên.

- Bạn cần một hàm chính `quick_sort(arr, low, high)` hoạt động đệ quy.
- Bạn cũng cần một hàm phụ `partition(arr, low, high)` để chọn một "chốt" (pivot), phân hoạch danh sách thành hai phần (nhỏ hơn pivot và lớn hơn pivot), và trả về chỉ số mới của pivot.
- Chọn pivot đơn giản nhất là phần tử cuối cùng (`arr[high]`).

Gợi ý hướng dẫn:

1. **Hàm `partition`:**
    - Chọn `pivot = arr[high]`.
    - Khởi tạo `i = low - 1` (chỉ số của phần tử nhỏ hơn cuối cùng).
    - Duyệt `j` từ `low` đến `high - 1`.
    - Nếu `arr[j] < pivot`, tăng `i` và hoán đổi `arr[i]` với `arr[j]`.
    - Sau vòng lặp, hoán đổi `arr[i + 1]` với `arr[high]` (đưa pivot về đúng vị trí).
    - Trả về `i + 1` (chỉ số của pivot).
2. **Hàm `quick_sort` (Đệ quy):**
    - Điều kiện dừng: `if low < high`.
    - Gọi `pi = partition(arr, low, high)`.
    - Gọi đệ quy cho hai nửa:
        - `quick_sort(arr, low, pi - 1)` (bên trái pivot)
        - `quick_sort(arr, pi + 1, high)` (bên phải pivot)

Code mẫu (Giải pháp tham khảo):

Python

```
def partition(arr: list, low: int, high: int) -> int:
    pivot = arr[high]
    i = low - 1  # Chỉ số của phần tử nhỏ hơn

    for j in range(low, high):
```

```
        if arr[j] < pivot:
            i += 1
            # Hoán đổi arr[i] và arr[j]
            arr[i], arr[j] = arr[j], arr[i]

    # Hoán đổi arr[i + 1] và pivot (arr[high])
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1  # Trả về chỉ số của pivot


def quick_sort(arr: list, low: int, high: int) -> None:
    if low < high:
        # pi là chỉ số phân hoạch
        pi = partition(arr, low, high)

        # Sắp xếp đệ quy 2 nửa
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)


if __name__ == "__main__":
    mang_so = [10, 7, 8, 9, 1, 5]
    print(f"Mang ban dau: {', '.join(map(str, mang_so))}")

    quick_sort(mang_so, 0, len(mang_so) - 1)

    print(f"Mang sau khi sap xep: {', '.join(map(str, mang_so))}")
```

Vận dụng AI gia tăng trải nghiệm:

- **Hỏi AI:** "Hãy giải thích chiến lược 'Chia để trị' (Divide and Conquer) mà Quick Sort sử dụng."
- **Thử thách Pythonic:** "Làm thế nào để viết Quick Sort cực kỳ ngắn gọn trong Python bằng cách dùng List Comprehension? Cách này có ưu/nhược điểm gì về bộ nhớ so với cách triển khai in-place?"
- **Phân tích:** "Độ phức tạp thời gian trường hợp tốt nhất, trung bình và xấu nhất của Quick Sort là gì? Trong Python, giới hạn độ sâu đệ quy (`recursion limit`) ảnh hưởng gì đến Quick Sort?"

Bài 5: Quản lý danh sách sinh viên

- **Mức độ:** Rất khó / Vận dụng

- **Thuật toán:** Sắp xếp đối tượng (Object Sorting) và Tìm kiếm

Mô tả yêu cầu chi tiết:

1. Tạo một lớp (class) `SinhVien` có các thuộc tính: `ma_sv` (str), `ho_ten` (str), `diem_tb` (float). Cung cấp phương thức `__str__` để định dạng hiển thị thông tin sinh viên.
2. Tạo một danh sách `ds_sinh_vien` và thêm vào đó ít nhất 5 sinh viên với dữ liệu mẫu.
3. **Yêu cầu 1 (Sắp xếp):** Viết code để sắp xếp danh sách này theo tiêu chí:
    - **Ưu tiên 1:** `diem_tb` giảm dần.
    - **Ưu tiên 2:** Nếu `diem_tb` bằng nhau, sắp xếp theo `ho_ten` tăng dần (theo thứ tự alphabet).
    - Sử dụng hàm `sorted()` hoặc phương thức `list.sort()` với tham số `key` kết hợp hàm `lambda`.
4. **Yêu cầu 2 (Tìm kiếm):** Viết một hàm `tim_sinh_vien(ds, ten)` để tìm và trả về đối tượng `SinhVien` đầu tiên có tên chính xác là `ten` (không phân biệt hoa/thường). Nếu không tìm thấy, trả về `None`.

Gợi ý hướng dẫn:

- **Yêu cầu 1 (Sắp xếp đa tiêu chí):** Python cho phép sử dụng `tuple` làm key trong `sorted()` hoặc `sort()`:

```python
ds_da_sap_xep = sorted(
    ds_sinh_vien, key=lambda sv: (-sv.diem_tb, sv.ho_ten)
)
```

(Lưu ý: `-sv.diem_tb` giúp đảo ngược thứ tự để sắp xếp giảm dần với điểm số).

- **Yêu cầu 2 (Tìm kiếm):** Duyệt qua danh sách và so sánh chuỗi viết thường:

```python
sv.ho_ten.lower() == ten.lower()
```

Code mẫu (Giải pháp tham khảo):

```python
class SinhVien:

    def __init__(self, ma_sv: str, ho_ten: str, diem_tb: float):
        self.ma_sv = ma_sv
        self.ho_ten = ho_ten
        self.diem_tb = diem_tb
```

```
    def __str__(self) -> str:
        return f"[{self.ma_sv}] {self.ho_ten} - Diem: {self.diem_tb}"


def tim_sinh_vien(ds: list, ten: str):
    # Tìm sinh viên đầu tiên trùng tên (không phân biệt hoa/thường)
    for sv in ds:
        if sv.ho_ten.lower() == ten.lower():
            return sv
    return None


if __name__ == "__main__":
    ds_sinh_vien = [
        SinhVien("SV001", "Nguyen Van An", 8.5),
        SinhVien("SV002", "Tran Thi Binh", 7.2),
        SinhVien("SV003", "Le Van Cuong", 8.5),
        SinhVien("SV004", "Pham Thi Dung", 9.1),
        SinhVien("SV005", "Hoang Van Em", 7.2),
    ]

    # Yêu cầu 1: Sắp xếp dùng sorted() với lambda key
    ds_da_sap_xep = sorted(
        ds_sinh_vien, key=lambda sv: (-sv.diem_tb, sv.ho_ten)
    )

    print("--- Danh sach sinh vien sau khi sap xep ---")
    for sv in ds_da_sap_xep:
        print(sv)

    # Yêu cầu 2: Tìm kiếm
    ten_can_tim = "Le Van Cuong"
    sv_tim_thay = tim_sinh_vien(ds_da_sap_xep, ten_can_tim)

    print("\n--- Ket qua tim kiem ---")
    if sv_tim_thay:
        print(f"Tim thay: {sv_tim_thay}")
    else:
        print(f"Khong tim thay sinh vien ten '{ten_can_tim}'.")
```

Vận dụng AI gia tăng trải nghiệm:

- **Hỏi AI:** "Hãy giải thích cơ chế hoạt động của tham số `key` và `lambda` trong hàm `sorted()` của Python. Tại sao dùng `-sv.diem_tb` lại sắp xếp giảm dần?"

- **Refactor:** "Hãy viết lại Yêu cầu 1 (Sắp xếp) bằng cách triển khai các phương thức so sánh dunder method (như `__lt__`, `__eq__`) ngay bên trong lớp `SinhVien` thay vì dùng lambda."
- **Hiệu năng:** "Nếu danh sách sinh viên có 1 triệu người, việc tìm kiếm bằng vòng lặp tuyến tính có hiệu quả không? Làm thế nào để dùng cấu trúc dữ liệu Dictionary (`dict`) trong Python để tra cứu sinh viên theo tên với độ phức tạp $O(1)$?"

Bài 6: Sắp xếp chọn (Selection Sort)

- **Mức độ:** Dễ / Trung bình
- **Thuật toán:** Sắp xếp chọn (Selection Sort)

Mô tả yêu cầu chi tiết:

Viết một hàm `sap_xep_chon(arr)` nhận đầu vào là một danh sách số nguyên và sắp xếp danh sách đó theo thứ tự tăng dần bằng thuật toán Selection Sort.

- Ý tưởng thuật toán: Duyệt qua từng vị trí `i` trong danh sách, tìm phần tử nhỏ nhất trong đoạn còn lại từ `i` đến cuối danh sách, sau đó đổi chỗ phần tử nhỏ nhất đó với phần tử ở vị trí `i`.
- Hàm thực hiện sắp xếp trực tiếp trên danh sách gốc (in-place).

Gợi ý hướng dẫn:

1. Dùng vòng lặp `i` chạy từ `0` đến `len(arr) - 2`.
2. Giả sử chỉ số của phần tử nhỏ nhất hiện tại là `min_idx = i`.
3. Dùng vòng lặp `j` chạy từ `i + 1` đến `len(arr) - 1`:
    - Nếu `arr[j] < arr[min_idx]`, cập nhật `min_idx = j`.
4. Sau khi kết thúc vòng lặp `j`, hoán đổi `arr[i]` và `arr[min_idx]` bằng cú pháp `arr[i], arr[min_idx] = arr[min_idx], arr[i]`.

Code mẫu (Giải pháp tham khảo):

Python

```python
def sap_xep_chon(arr: list) -> None:
    n = len(arr)
    for i in range(n - 1):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        # Đổi chỗ phần tử nhỏ nhất tìm được với vị trí i hiện tại
```

```
        arr[i], arr[min_idx] = arr[min_idx], arr[i]


if __name__ == "__main__":
    mang_so = [29, 10, 14, 37, 13, 8, 24]
    print(f"Mang ban dau: {mang_so}")
    sap_xep_chon(mang_so)
    print(f"Mang sau khi sap xep (Selection Sort): {mang_so}")
```

Vận dụng AI gia tăng trải nghiệm:

- **Hỏi AI:** "Số lần hoán đổi (swap) của Selection Sort là bao nhiêu? Tại sao Selection Sort lại hữu ích khi chi phí thực hiện phép hoán đổi bộ nhớ (write operation) là rất tốn kém?"
- **So sánh:** "So sánh số phép so sánh và số phép gán giữa Bubble Sort và Selection Sort trong trường hợp xấu nhất."

Bài 7: Sắp xếp chèn (Insertion Sort)

- **Mức độ:** Dễ / Trung bình
- **Thuật toán:** Sắp xếp chèn (Insertion Sort)

Mô tả yêu cầu chi tiết:

Viết một hàm `sap_xep_chen(arr)` thực hiện sắp xếp danh sách số nguyên theo thứ tự tăng dần bằng thuật toán Insertion Sort.

- Thuật toán hoạt động tương tự như cách sắp xếp các lá bài trên tay: Xem phần tử đầu tiên đã được sắp xếp. Với mỗi phần tử tiếp theo, tìm vị trí thích hợp và "chèn" nó vào dãy đã sắp xếp ở phía trước.

Gợi ý hướng dẫn:

1. Duyệt vòng lặp `i` từ `1` đến `len(arr) - 1`.
2. Lưu giá trị phần tử cần chèn vào biến `key = arr[i]`.
3. Khởi tạo biến chỉ số `j = i - 1`.
4. Dùng vòng lặp `while j >= 0 và arr[j] > key`:
    - Dịch chuyển `arr[j]` sang phải một vị trí: `arr[j + 1] = arr[j]`.
    - Giảm `j` xuống 1 đơn vị (`j -= 1`).
5. Gán giá trị `key` vào vị trí đúng: `arr[j + 1] = key`.

Code mẫu (Giải pháp tham khảo):

```python
def sap_xep_chen(arr: list) -> None:
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        # Di chuyển các phần tử lớn hơn key sang phải một vị trí
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key


if __name__ == "__main__":
    mang_so = [12, 11, 13, 5, 6, 7]
    print(f"Mang ban dau: {mang_so}")
    sap_xep_chen(mang_so)
    print(f"Mang sau khi sap xep (Insertion Sort): {mang_so}")
```

Vận dụng AI gia tăng trải nghiệm:

- **Hỏi AI:** "Tại sao Insertion Sort lại hoạt động cực kỳ nhanh đối với mảng gần như đã được sắp xếp (Nearly Sorted Array) với độ phức tạp xấp xỉ $O(n)$?"
- **Ứng dụng thực tế:** "Hỏi AI về thuật toán `Timsort` (thuật toán sắp xếp mặc định của Python) và cách nó kết hợp giữa Merge Sort và Insertion Sort."

Bài 8: Sắp xếp trộn (Merge Sort)

- **Mức độ:** Khó
- **Thuật toán:** Sắp xếp trộn (Merge Sort)

Mô tả yêu cầu chi tiết:

Viết chương trình triển khai thuật toán Merge Sort bằng phương pháp đệ quy để sắp xếp danh sách số nguyên theo thứ tự tăng dần.

- Ý tưởng:
    1. **Chia (Divide):** Chia danh sách thành 2 nửa bằng nhau.
    2. **Trị (Conquer):** Đệ quy sắp xếp 2 danh sách con.

3. **Kết hợp (Combine):** Trộn (Merge) 2 danh sách con đã sắp xếp thành danh sách hoàn chỉnh.

Gợi ý hướng dẫn:

1. Điều kiện cơ sở của đệ quy: Nếu `len(arr) <= 1`, trả về `arr`.
2. Tìm vị trí giữa `mid = len(arr) // 2`.
3. Chia danh sách thành 2 nửa: `left = arr[:mid]` và `right = arr[mid:]`.
4. Gọi đệ quy cho 2 nửa: `left_sorted = merge_sort(left)` và `right_sorted = merge_sort(right)`.
5. Viết một hàm phụ `merge(left, right)` để trộn 2 danh sách đã sắp xếp lại với nhau theo thứ tự tăng dần.

Code mẫu (Giải pháp tham khảo):

Python

```
def merge(left: list, right: list) -> list:
    result = []
    i = j = 0

    # Trộn 2 danh sách cho đến khi 1 trong 2 hết phần tử
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    # Thêm các phần tử còn lại (nếu có)
    result.extend(left[i:])
    result.extend(right[j:])
    return result


def merge_sort(arr: list) -> list:
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])

    return merge(left_half, right_half)


if __name__ == "__main__":
```

```
mang_so = [38, 27, 43, 3, 9, 82, 10]
print(f"Mang ban dau: {mang_so}")
mang_da_sap_xep = merge_sort(mang_so)
print(f"Mang sau khi sap xep (Merge Sort): {mang_da_sap_xep}")
```

Vận dụng AI gia tăng trải nghiệm:

- **Hỏi AI:** "Giải thích tại sao Merge Sort luôn giữ độ phức tạp thời gian $O(n \log n)$ trong mọi trường hợp (tốt nhất, trung bình, xấu nhất)."
- **Phân tích bộ nhớ:** "So sánh nhược điểm về bộ nhớ phụ ($O(n)$ space complexity) của Merge Sort so với Quick Sort ($O(\log n)$ space)."

Bài 9: Tìm phần tử nhỏ thứ K (QuickSelect)

- **Mức độ:** Khó
- **Thuật toán:** QuickSelect (Ứng dụng tư tưởng Quick Sort)

Mô tả yêu cầu chi tiết:

Cho một danh sách số nguyên chưa sắp xếp và một số nguyên dương `k`. Hãy viết hàm `quick_select(arr, k)` để tìm phần tử nhỏ thứ `k` trong danh sách (chỉ số tính từ 1) **mà không cần sắp xếp toàn bộ danh sách**.

- *Ví dụ:* Mảng `[7, 10, 4, 3, 20, 15]`, với `k = 3`, 3 phần tử nhỏ nhất là `3, 4, 7` → Kết quả trả về là `7`.

Gợi ý hướng dẫn:

1. Sử dụng hàm `partition(arr, low, high)` tương tự như Quick Sort.
2. Sau khi `partition` trả về chỉ số của pivot là `pi`:
    - Nếu `pi == k - 1`, ta tìm thấy phần tử cần tìm tại `arr[pi]`.
    - Nếu `pi > k - 1`, ta tiếp tục tìm đệ quy ở nửa bên trái: `quick_select(arr, low, pi - 1, k)`.
    - Nếu `pi < k - 1`, ta tiếp tục tìm đệ quy ở nửa bên phải: `quick_select(arr, pi + 1, high, k)`.

Code mẫu (Giải pháp tham khảo):

```python
def partition(arr: list, low: int, high: int) -> int:
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1


def quick_select_helper(arr: list, low: int, high: int, k_index: int) -> int:
    if low <= high:
        pi = partition(arr, low, high)

        if pi == k_index:
            return arr[pi]
        elif pi > k_index:
            return quick_select_helper(arr, low, pi - 1, k_index)
        else:
            return quick_select_helper(arr, pi + 1, high, k_index)
    return -1


def tim_phan_tu_nho_thu_k(arr: list, k: int) -> int:
    # k_index là chỉ số 0-based
    if 1 <= k <= len(arr):
        # Tạo bản sao để tránh làm thay đổi mảng gốc nếu cần
        arr_copy = arr.copy()
        return quick_select_helper(arr_copy, 0, len(arr_copy) - 1, k - 1)
    raise ValueError("Gia tri k khong hop le!")


if __name__ == "__main__":
    mang_so = [7, 10, 4, 3, 20, 15]
    k = 3
    ket_qua = tim_phan_tu_nho_thu_k(mang_so, k)
    print(f"Phan tu nho thu {k} trong mang {mang_so} la: {ket_qua}")
```

Vận dụng AI gia tăng trải nghiệm:

- **Hỏi AI:** "Tại sao độ phức tạp thời gian trung bình của QuickSelect chỉ là $O(n)$, trong khi sắp xếp mảng rồi lấy phần tử thứ k tốn $O(n \log n)$?"

- **Thử thách:** "Làm thế nào để áp dụng thư viện `heapq` trong Python để giải bài toán Top K Elements?"

Bài 10: Tra cứu và lọc sản phẩm e-Commerce

- **Mức độ:** Rất khó / Vận dụng thực tế
- **Thuật toán:** Sắp xếp danh sách đối tượng & Tìm kiếm nhị phân khoảng giá (Binary Search Range)

Mô tả yêu cầu chi tiết:

Xây dựng hệ thống tìm kiếm sản phẩm cho website thương mại điện tử:

1. Định nghĩa lớp `SanPham` gồm: `ma_sp` (str), `ten_sp` (str), `gia` (float).
2. Viết hàm `sap_xep_theo_gia(ds_san_pham)` để sắp xếp danh sách sản phẩm theo giá tăng dần.
3. Viết hàm `tim_san_pham_theo_khoang_gia(ds_da_sap_xep, gia_min, gia_max)` triển khai bằng **Tìm kiếm nhị phân** để lọc và trả về tất cả các sản phẩm có giá nằm trong khoảng `[gia_min, gia_max]`.

Gợi ý hướng dẫn:

1. Sau khi danh sách đã được sắp xếp tăng dần theo giá:
2. Dùng Tìm kiếm nhị phân biến thể (Lower Bound) để tìm **chỉ số đầu tiên** có `gia >= gia_min`.
3. Dùng Tìm kiếm nhị phân biến thể (Upper Bound) để tìm **chỉ số cuối cùng** có `gia <= gia_max`.
4. Trả về danh sách con từ chỉ số đầu đến chỉ số cuối bằng kỹ thuật Python Slicing (`arr[start:end+1]`).
5. Có thể vận dụng module `bisect` có sẵn trong thư viện chuẩn Python.

Code mẫu (Giải pháp tham khảo):

Python

```python
import bisect


class SanPham:

    def __init__(self, ma_sp: str, ten_sp: str, gia: float):
        self.ma_sp = ma_sp
        self.ten_sp = ten_sp
```

```
        self.gia = gia

    def __str__(self) -> str:
        return f"[{self.ma_sp}] {self.ten_sp} - Gia: {self.gia:,.0f} VND"


def tim_san_pham_theo_khoang_gia(
    ds_da_sap_xep: list, gia_min: float, gia_max: float
) -> list:
    # Trích xuất danh sách giá để dùng với thư viện bisect
    cac_muc_gia = [sp.gia for sp in ds_da_sap_xep]

    # Tìm chỉ số đầu tiên có giá >= gia_min
    left_idx = bisect.bisect_left(cac_muc_gia, gia_min)

    # Tìm chỉ số đầu tiên có giá > gia_max
    right_idx = bisect.bisect_right(cac_muc_gia, gia_max)

    # Trả về danh sách sản phẩm nằm trong khoảng giá
    return ds_da_sap_xep[left_idx:right_idx]


if __name__ == "__main__":
    ds_sp = [
        SanPham("SP01", "Chuot Khong Day", 250000),
        SanPham("SP02", "Ban Phim Co", 1200000),
        SanPham("SP03", "Tai Nghe Bluetooth", 850000),
        SanPham("SP04", "Man Hinh 24 inch", 3500000),
        SanPham("SP05", "Lot Chuot RGB", 150000),
        SanPham("SP06", "Webcam HD", 650000),
    ]

    # Bước 1: Sắp xếp danh sách theo giá tăng dần
    ds_sp_sap_xep = sorted(ds_sp, key=lambda sp: sp.gia)

    print("--- DANH SÁCH SẢN PHẨM ĐÃ SẮP XẾP THEO GIÁ ---")
    for sp in ds_sp_sap_xep:
        print(sp)

    # Bước 2: Tìm sản phẩm trong khoảng giá 200,000 đến 1,000,000 VND
    gia_tu = 200000
    gia_den = 1000000
    ket_qua = tim_san_pham_theo_khoang_gia(ds_sp_sap_xep, gia_tu, gia_den)

    print(f"\n--- SẢN PHẨM TRONG KHOẢNG GIÁ {gia_tu:,.0f} - {gia_den:,.0f} VND
---")
    if ket_qua:
        for sp in ket_qua:
```

```
        print(sp)
else:
    print("Khong co san pham nao trong khoang gia nay.")
```

Vận dụng AI gia tăng trải nghiệm:

- **Hỏi AI:** "Giải thích nguyên lý hoạt động của `bisect_left` và `bisect_right` trong thư viện `bisect` của Python. Độ phức tạp tìm kiếm khoảng bằng cách này là bao nhiêu?"
- **Thử thách nâng cao:** "Hãy tự viết hàm `bisect_left` bằng tay (không dùng module `bisect`) để hiểu rõ cách điều chỉnh điều kiện trong Binary Search."